

Tipos de Control de Acceso

ACL, RBAC, ABAC, PBAC, RBAC jerárquico y ReBAC — guía completa con ejemplos de código

Los sistemas de control de acceso deciden **quién puede hacer qué** dentro de una aplicación. Elegir el correcto depende de la complejidad de las reglas de negocio y la escala del proyecto.

1. ACL — Access Control List

■ Usado en: Google Drive, Dropbox, sistemas de archivos

El más simple. Una lista que dice exactamente qué usuario tiene acceso a qué recurso concreto.

```
const permisos = {
  'documento-123': ['ana', 'juan'],
  'documento-456': ['ana', 'maria'],
  'documento-789': ['todos']
}

function puedeAcceder(usuario: string, recurso: string): boolean {
  return permisos[recurso].includes(usuario)
}
```

- ■ **Simple** de implementar para casos pequeños
- ■ **No escala** — con 1000 usuarios y 500 documentos la lista es inmanejable

2. RBAC — Role Based Access Control

■ Usado en: WordPress, sistemas de gestión, e-commerce con panel de administración

El más común en aplicaciones web. Los permisos se asignan a **roles** y los usuarios tienen roles.

```
const roles = {
  admin: ['crear', 'leer', 'editar', 'eliminar'],
  editor: ['crear', 'leer', 'editar'],
  viewer: ['leer'],
}

function tienePermiso(usuario, accion: string): boolean {
  return roles[usuario.rol].includes(accion)
}

tienePermiso(usuario, 'editar') // ■ true (si rol = editor)
tienePermiso(usuario, 'eliminar') // ■ false (si rol = editor)
```

En Next.js con NextAuth

```
// middleware.ts
export function middleware(request: NextRequest) {
  const token = await getToken({ req: request })

  if (request.nextUrl.pathname.startsWith('/admin')) {
    if (token?.rol !== 'admin') {
      return NextResponse.redirect(new URL('/unauthorized', request.url))
    }
  }
}
```

```
}  
}
```

En Prisma

```
enum Rol {  
  ADMIN  
  EDITOR  
  VIEWER  
}  
  
model Usuario {  
  id      String @id @default(cuid())  
  nombre  String  
  email   String @unique  
  rol     Rol    @default(VIEWER)  
}
```

- ■ **Equilibrio** perfecto entre simplicidad y potencia
- ■ **Poco flexible** cuando las reglas son muy específicas por usuario

3. ABAC — Attribute Based Access Control

■ Usado en: Sistemas bancarios, gobierno, sanidad — reglas complejas

Más flexible que RBAC. Los permisos dependen de **atributos** del usuario, del recurso y del contexto.

```
function puedeEditar(usuario, documento, contexto): boolean {  
  return (  
    usuario.rol === 'editor' &&  
    documento.autorId === usuario.id && // solo tu contenido  
    documento.estado !== 'publicado' && // no si ya publicado  
    contexto.hora >= 9 && contexto.hora <= 18 // horario laboral  
  )  
}
```

Ejemplo real — médico y pacientes

```
function puedeVerHistorial(medico, paciente): boolean {  
  return (  
    medico.especialidad === paciente.especialidadRequerida &&  
    medico.pacientes.includes(paciente.id) &&  
    medico.turnoActivo === true  
  )  
}
```

- ■ **Muy flexible** — puede expresar cualquier regla de negocio
- ■ **Complejo** de implementar y mantener

4. PBAC — Permission Based Access Control

■ Usado en: APIs con clientes distintos, sistemas SaaS con permisos granulares

En lugar de roles, asignas **permisos individuales** directamente a cada usuario. Más granular que RBAC.

```
const usuario = {  
  id: '123',  
  permisos: ['productos:leer', 'productos:editar', 'pedidos:leer']  
}  
  
function tienePermiso(usuario, permiso: string): boolean {  
  return usuario.permisos.includes(permiso)  
}
```

```
tienePermiso(usuario, 'productos:editar') // ■ true
tienePermiso(usuario, 'usuarios:eliminar') // ■ false
```

- ■ **Granular** — cada usuario puede tener una combinación única
- ■ **Costoso** de gestionar con muchos usuarios

5. RBAC con jerarquía (Role Hierarchy)

■ Usado en: CMS, ERPs, plataformas con múltiples niveles de administración

Extensión de RBAC donde los roles **heredan permisos** de roles inferiores.

```
const jerarquia = {
  superAdmin: ['admin', 'editor', 'viewer'],
  admin:      ['editor', 'viewer'],
  editor:     ['viewer'],
  viewer:     []
}

const permisos = {
  superAdmin: ['gestionar-sistema'],
  admin:      ['eliminar', 'gestionar-usuarios'],
  editor:     ['crear', 'editar'],
  viewer:     ['leer']
}

function obtenerTodosLosPermisos(rol: string): string[] {
  const rolesHeredados = [rol, ...jerarquia[rol]]
  return rolesHeredados.flatMap(r => permisos[r])
}

obtenerTodosLosPermisos('admin')
// → ['eliminar', 'gestionar-usuarios', 'crear', 'editar', 'leer']
```

- ■ **Intuitivo** — los roles superiores tienen más permisos automáticamente
- ■ **Puede volverse** complejo con muchos niveles de herencia

6. ReBAC — Relationship Based Access Control

■ Usado en: Google Docs, Notion, Figma, sistemas de colaboración

Los permisos dependen de la **relación** entre el usuario y el recurso. Google lo usa internamente.

```
function puedeAcceder(usuario, documento): boolean {
  return (
    documento.autorId === usuario.id ||           // es el autor
    usuario.equipoId === documento.equipoId ||    // mismo equipo
    documento.compartidoCon.includes(usuario.id) // compartido
  )
}
```

- ■ **Natural** para apps colaborativas — refleja relaciones reales
- ■ **Complejo** de implementar correctamente a escala

Tabla comparativa

Sistema	Complejidad	Escalabilidad	Flexibilidad	Caso típico
---------	-------------	---------------	--------------	-------------

ACL	Baja	■ Mala	Baja	Compartir archivos
RBAC	Media	■ Buena	Media	Apps web típicas
ABAC	Alta	■ Buena	Muy alta	Sanidad, banca
PBAC	Media	■ Buena	Alta	APIs, SaaS
RBAC jerárquico	Media	■ Buena	Media-alta	CMS, ERPs
ReBAC	Alta	■ Buena	Alta	Herramientas colaborativas

¿Cuál usar en tu proyecto?

Para la mayoría de aplicaciones web la respuesta es **RBAC** — es el equilibrio perfecto entre simplicidad y potencia. Solo necesitas algo más complejo si tienes reglas de negocio muy específicas.

App sencilla con admin/usuario	→ RBAC básico
E-commerce con varios roles	→ RBAC jerárquico
App con permisos muy granulares	→ PBAC
App colaborativa tipo Notion	→ ReBAC
Sistema crítico con muchas reglas	→ ABAC